

We're ready to start Lesson 5.1 in the new chat.



Next.js Lesson 5.1 — Mutations & User Interactions

Yep — Lesson 4 is officially complete. 🏁

Now we move into the part of Next.js where the app stops being mostly:

User → page → server fetch → render

and becomes:

User → interaction → change data → update UI

This is a *major* shift in how you think about Next.js.



Lesson 5 Roadmap

We're going to learn:

5.1 — Mutations: changing data

- What is a mutation?
- Why fetching ≠ mutating
- Server-side mutations vs client-side mutations
- Forms and submissions
- Where mutation code should live

5.2 — Server Actions

- What Server Actions actually are
- `"use server"`
- Calling a Server Action from a form
- Arguments and return values
- Validation
- Error handling

5.3 — Revalidation after mutations

- Why the UI can become stale
- `revalidatePath`

- `revalidateTag`
- How mutation + cache invalidation work together

5.4 — Client-side interactions

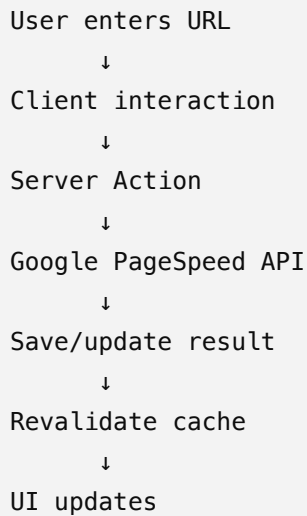
- `"use client"`
- When you actually need a Client Component
- Event handlers
- Loading/submitting states
- `useTransition`

5.5 — Optimistic UI

- Why waiting for the server can feel slow
- `useOptimistic`
- Showing the expected result immediately
- Handling failure/rollback conceptually

5.6 — Real architecture

We'll put everything together in your **Web Performance Analyzer**:



```
graph TD; A[User enters URL] --> B[Client interaction]; B --> C[Server Action]; C --> D[Google PageSpeed API]; D --> E[Save/update result]; E --> F[Revalidate cache]; F --> G[UI updates];
```

And we'll specifically clarify the thing you've been wrestling with:

"If the server can do everything, why do I need client code at all?"

That's going to become much clearer here.

5.1 — What is a Mutation?

So far we've mostly dealt with **reading data**.

For example:

```
const data = await getPerformanceData();
```

That's a **query/read**.

You're saying:

"Server, give me some data."

A **mutation** is the opposite direction:

"Server, change something."

For example:

```
CREATE
UPDATE
DELETE
```



Example

Imagine a database containing:

Users



```
-----
id  name
1   Gopi
```

Reading:

```
GET user
  ↓
Gopi
```



Mutation:

```
UPDATE user
name = "Chandana"
```



Now the database changes:

Users



```
-----
id  name
1   Chandana
```

Query vs Mutation

Think of it like this:

Operation	Meaning
Query	Give me data
Mutation	Change data

Examples:

```
GET      → read
POST     → create
PUT/PATCH → update
DELETE   → delete
```



But here's an important Next.js point:

Mutation doesn't necessarily mean HTTP POST.

Mutation is a **concept**.

HTTP methods are one way of implementing that concept.

Your Performance Analyzer Example

Suppose your app has:

```
Web Performance Analyzer
URL: [ example.com ]
[ Analyze ]
```



User clicks:

Analyze

Now something needs to happen.

The application needs to:



```
User clicks Analyze
      ↓
Send URL to server
      ↓
Server calls Google PageSpeed
      ↓
Receive performance result
      ↓
Return result
      ↓
Display result
```

That operation is **not just a read anymore**.

The user has initiated an operation that causes work/state changes.

But here's an important distinction

Don't make this mistake:

"Everything involving a button is a mutation."

Not necessarily.

Suppose you have:

```
<button onClick={refreshResults}>
```

and `refreshResults()` simply fetches existing data.

That's still fundamentally a **read/query**.

Whereas:



```
Create report
Save report
Delete report
Update settings
Submit form
```

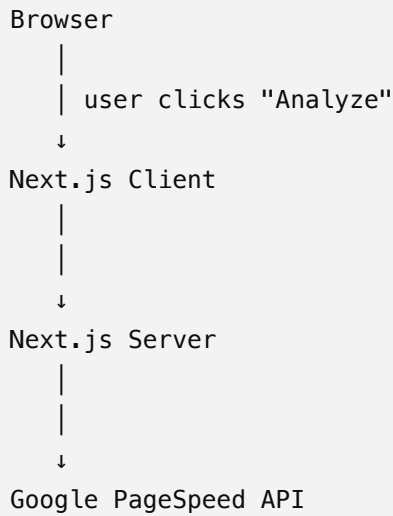
are mutations.



Where does the mutation happen?

This is where Next.js becomes interesting.

Imagine:



The **actual mutation/business operation** should generally happen on the server when it needs secrets, privileged access, database access, or trusted business logic.

The browser might initiate it, but the server performs the protected operation.

This is one of the core ideas we'll build on throughout Lesson 5.

⚠ The key mental model

Don't think:

```
Client = does the work
Server = does the work
```

Instead think:

```
CLIENT
____
"I want this to happen."

↓

SERVER
____
"Okay. I'll perform the trusted operation."

↓
```

DATABASE / EXTERNAL API

The client is often responsible for **interaction**.

The server is often responsible for **authority/business logic/data mutation**.

And this connects directly to your earlier question

You were asking:

"Why can't we manage everything from the server?"

You **can** put a huge amount of application logic on the server.

But there's one unavoidable fact:

The server cannot physically click a button for the user. 😊

The user is sitting in:

Browser



and does:

click
type
select
drag
submit



Something running in the browser has to respond to those interactions.

That's where **Client Components** come in.

But that **doesn't** mean your business logic has to move into the client.

For example:

```
CLIENT
button click
  ↓
  "Please analyze this URL"
    ↓
  SERVER
```



```
validate URL
  ↓
call PageSpeed
  ↓
process result
  ↓
return result
```

That's the architecture we're going to build toward.

First big principle of Lesson 5

Client interaction does not mean client-side business logic.

You can have:

```
Client
  ↓
trigger
  ↓
Server
  ↓
business logic
  ↓
external API / database
```



And **Server Actions** are one of the major Next.js mechanisms that make this communication much easier.

That's our next concept in **5.2**.

Before we jump there, make sure this distinction is solid:

```
QUERY
"I want to READ something."

MUTATION
"I want to CHANGE / PERFORM something."

CLIENT
"User interacted with the UI."

SERVER
"Perform trusted application work."
```



Next up: **why Server Actions exist and what "use server" actually does.** 🔥



ChatGPT can make mistakes. Check important info.